



Chatbots Avanzados con Retrieval-Augmented Generation

Esta presentación explora la evolución de los chatbots, desde sistemas basados en reglas hasta soluciones modernas con LLMs y RAG.



by Erasmo Gómez Montoya

¿Qué es un chatbot?

Un chatbot es un programa informático diseñado para simular conversaciones humanas.

Funciona como interfaz entre personas y sistemas digitales mediante texto o voz.



Conversacionales

Interpretan lenguaje natural y mantienen diálogos estructurados con usuarios.



Automatizados

Responden sin intervención humana mediante algoritmos predefinidos.



Asistenciales

Ayudan en tareas específicas como responder preguntas o realizar acciones.

Historia breve de los chatbots



Tipos de chatbots

Basados en reglas

- Utilizan árboles de decisión predefinidos
- Respuestas limitadas a patrones programados
- No aprenden de interacciones
- Implementación sencilla y predecible



Precisión en tareas definidas

Chatbots basados en reglas

Basados en IA

- Procesan lenguaje natural (NLP)
- Generan respuestas dinámicas
- Aprenden de conversaciones previas
- Manejan consultas complejas e imprevistas



Versatilidad de respuestas

Chatbots basados en IA

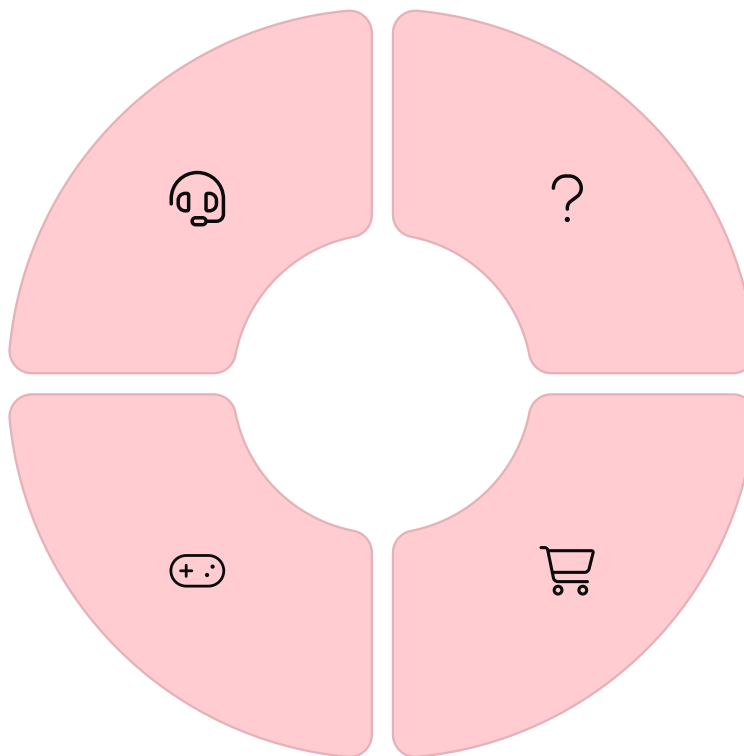
Casos de uso tradicionales

Servicio al cliente

- Respuesta a consultas frecuentes
- Gestión de tickets de soporte
- Disponibilidad 24/7

Entretenimiento

- Juegos conversacionales
- Compañeros virtuales



FAQ automatizadas

- Resolución de dudas comunes
- Reducción de carga a personal

Asistencia en compras

- Recomendaciones de productos
- Guía en proceso de compra

Arquitectura clásica de chatbots



Entrada del usuario

El usuario envía un mensaje en lenguaje natural al sistema.



Reconocimiento de intents

El sistema identifica la intención detrás del mensaje.



Procesamiento NLP

Extrae entidades y contexto relevante del mensaje.



Flujo de decisión

Determina la ruta de conversación según reglas predefinidas.



Generación de respuesta

Crea y envía una respuesta apropiada al usuario.

Evolución del NLP en chatbots

Expresiones regulares (1990s)

Patrones simples para identificar palabras clave en textos.

Clasificación de texto (2000s)

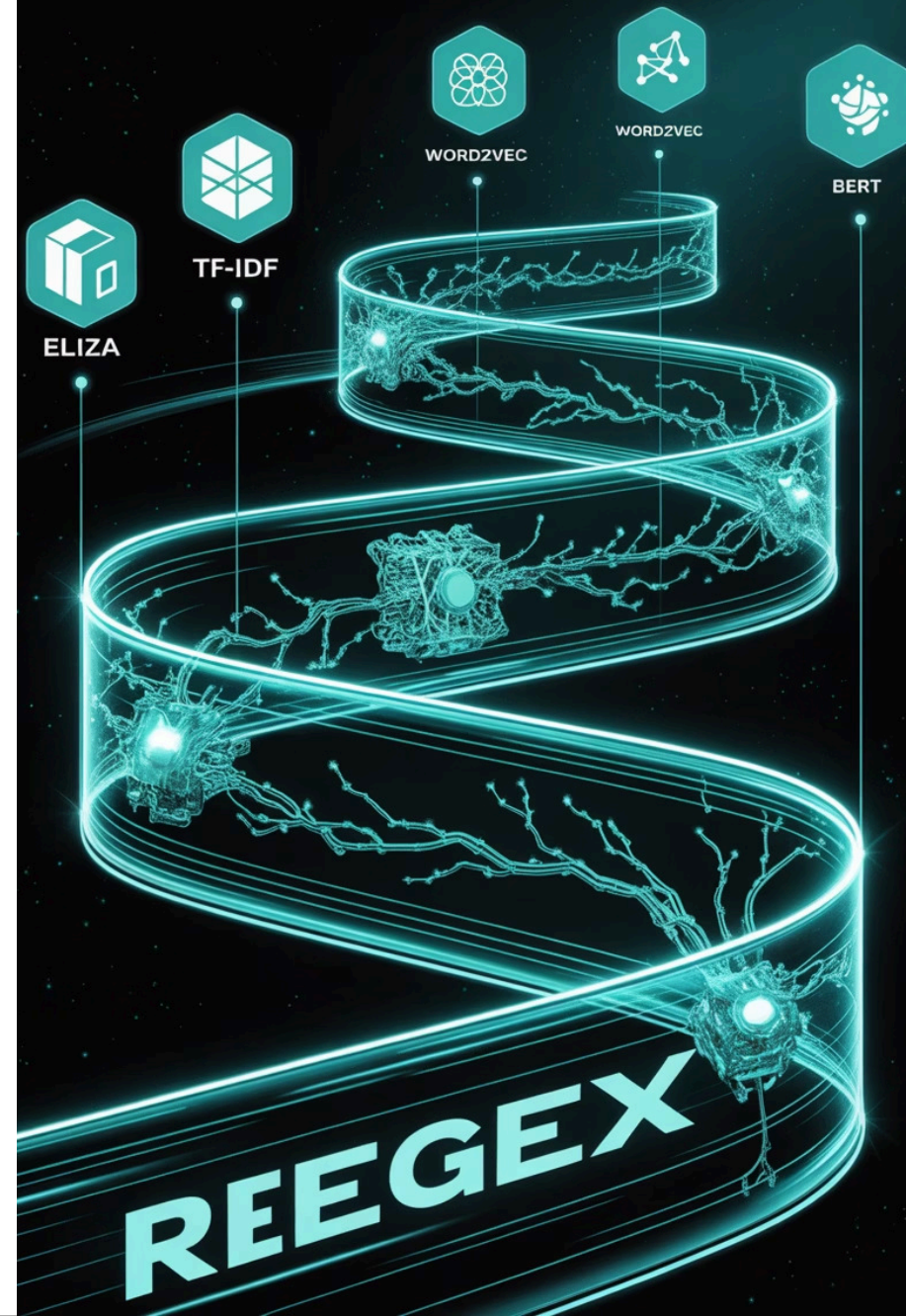
Uso de bolsa de palabras (BoW) y TF-IDF para categorizar consultas.

Word Embeddings (2013+)

Word2Vec y GloVe permitieron representar palabras como vectores semánticos.

Transformers (2017+)

Revolucionaron el NLP con mecanismos de atención y procesamiento paralelo.



Chatbots con NLU

Comprensión del Lenguaje Natural

NLU permite a los chatbots **extraer significado** e **intención** de mensajes con formato libre.

Representa un salto cualitativo frente a sistemas basados en palabras clave.



Reconocimiento de intenciones

Identifica qué quiere lograr el usuario con su consulta.



Extracción de entidades

Detecta nombres, fechas, cantidades y otros datos clave.



Análisis de sentimiento

Evalúa la actitud emocional del usuario durante la conversación.



Chatbots con ML clásico

Naive Bayes

Clasificador probabilístico basado en el teorema de Bayes.

- Eficiente con datos limitados
- Ideal para clasificación de intenciones simples

Support Vector Machines

Algoritmo que encuentra hiperplanos óptimos para separar clases.

- Alta precisión en espacios de alta dimensionalidad
- Efectivo con conjuntos de datos medianos

Random Forest

Ensamble de árboles de decisión para clasificación robusta.

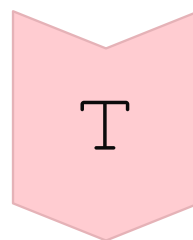
- Resistente al sobreajuste
- Maneja bien características no lineales

Embeddings y modelos secuenciales

Redes Neuronales Recurrentes

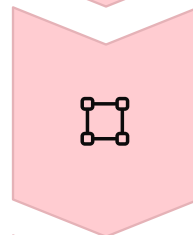
Las RNN y LSTM revolucionaron el procesamiento de secuencias de texto para chatbots.

- Memoria a corto plazo para contexto conversacional
- Capacidad de procesar texto como secuencia
- Mejora sustancial en comprensión contextual



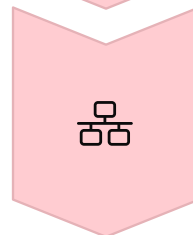
Tokenización

División del texto en unidades (tokens)



Embedding

Conversión a vectores numéricos



Procesamiento LSTM

Análisis secuencial con memoria

Transformer

Weight Weihant



Transformers: La revolución del NLP



Procesamiento paralelo

A diferencia de RNNs, procesan todas las palabras simultáneamente, acelerando entrenamiento e inferencia.



Mecanismo de atención

Permite al modelo "enfocarse" en diferentes partes de la entrada según su relevancia contextual.

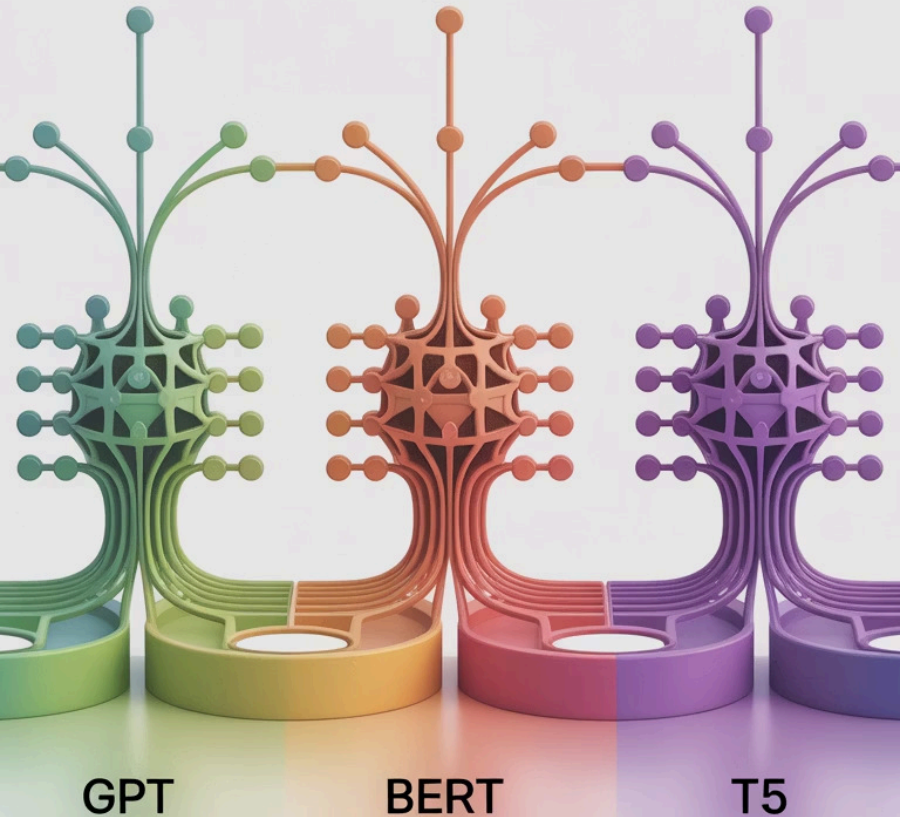


Contexto bidireccional

Considera palabras anteriores y posteriores para capturar mejor el significado completo del texto.

Introducidos en 2017 con "Attention is All You Need", los Transformers han redefinido el estado del arte en NLP.

Language Model Architectures



Modelos generativos pre-entrenados

GPT (Generative Pre-trained Transformer)

Modelo autoregresivo que predice tokens futuros en una secuencia.

- Ideal para generación de texto fluido y coherente
- Evolución desde GPT-1 hasta GPT-4 con capacidades mejoradas

BERT (Bidirectional Encoder)

Modelo bidireccional que comprende contexto en ambas direcciones.

- Óptimo para comprensión de consultas
- Puede adaptarse con capas adicionales (adapters)

T5 (Text-to-Text Transfer Transformer)

Unifica tareas de NLP como traducción, resumen y preguntas-respuestas.

- Versátil para múltiples funciones en chatbots
- Framework unificado para tareas diversas

LLMs como backends conversacionales

Ventajas de LLMs en chatbots

- Conocimiento general extenso
- Capacidad de mantener **contexto** en **conversaciones largas**
- Flexibilidad para adaptarse a múltiples dominios
- Respuestas **más naturales y humanas**

175B

Parámetros en GPT-3

Capacidad para capturar
patrones lingüísticos complejos

10x

Menos código

Reducción en desarrollo vs.
chatbots tradicionales

90%

Satisfacción

Usuarios prefieren chatbots
basados en LLMs

Prompt Engineering para chatbots

1 Definir rol y contexto

Establecer claramente quién es el chatbot y qué conocimientos debe tener.

Eres un asistente técnico especializado en Python que ayuda a desarrolladores.

2 Estructurar el formato de respuesta

Indicar cómo debe organizar y presentar la información.

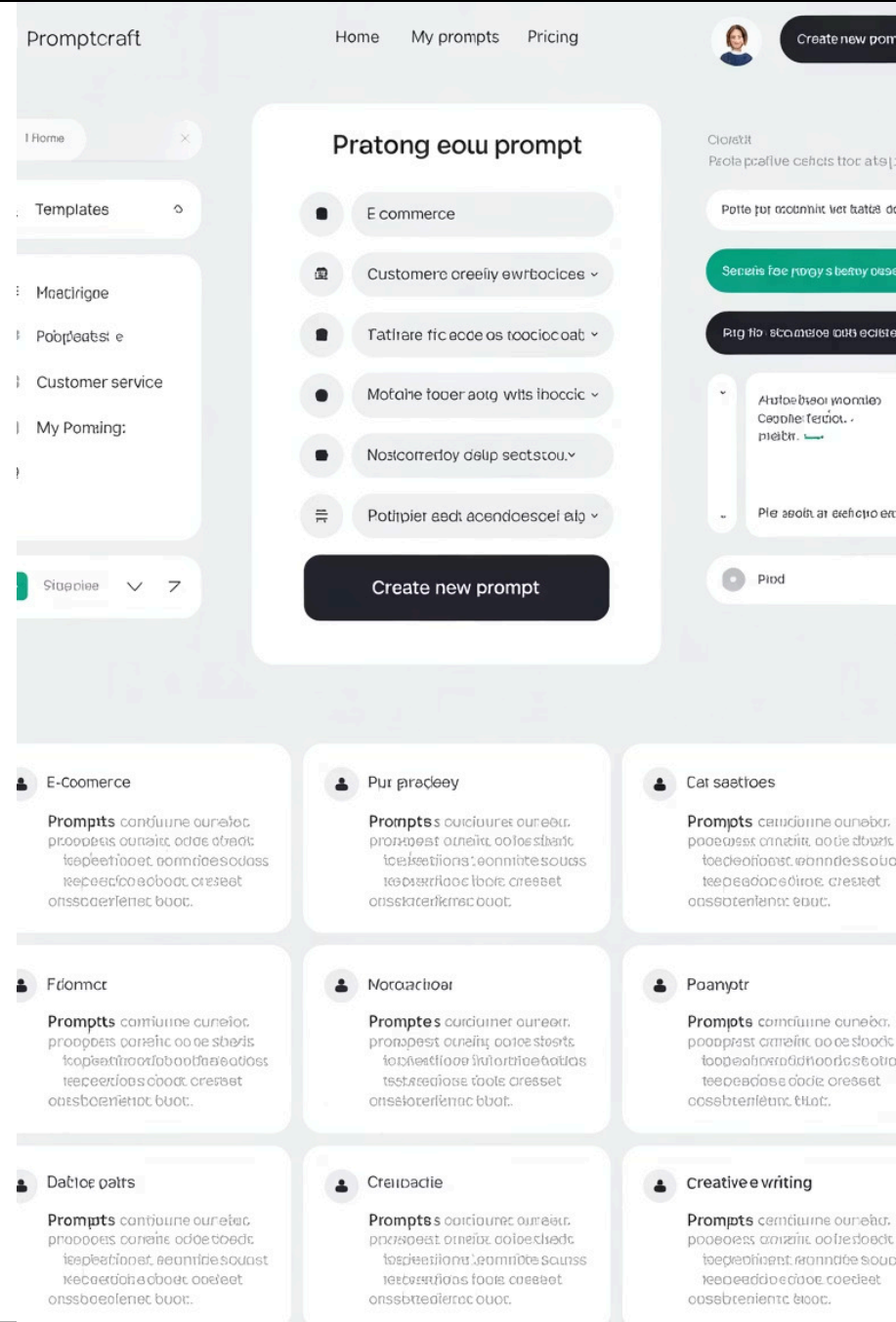
Responde en formato JSON con campos: solución, explicación, ejemplo.

3 Incluir ejemplos (few-shot)

Proporcionar ejemplos de interacciones ideales para guiar el comportamiento.

4 Establecer restricciones

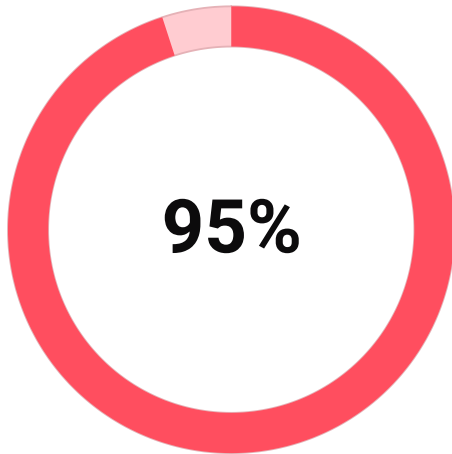
Definir límites claros sobre qué puede o no puede hacer el chatbot.



Fine-tuning vs. Few-shot Learning

Fine-tuning

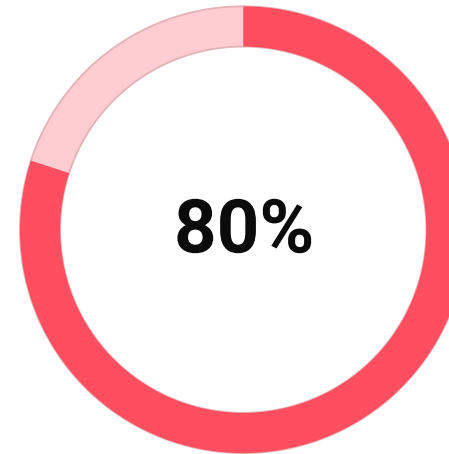
- Reentrenamiento del modelo con datos específicos
- Requiere conjuntos de datos etiquetados
- Mayor inversión de recursos computacionales
- Resultados óptimos para dominios específicos
- Permanente: cambia los pesos del modelo



Rendimiento potencial con fine-tuning específico

Few-shot Learning

- Aprendizaje mediante ejemplos en el prompt
- No requiere reentrenamiento del modelo
- Implementación rápida y flexible
- Menor precisión que fine-tuning especializado
- Temporal: solo afecta la inferencia actual



Rendimiento típico con few-shot bien diseñado

Ventajas y riesgos de LLMs como chatbots

Ventajas



Versatilidad

Adaptan respuestas a diversos contextos sin programación específica.



Comprensión robusta

Interpretan consultas ambiguas o con errores gramaticales.



Conversaciones naturales

Generan respuestas que fluyen como diálogos humanos reales.

Riesgos



Alucinaciones

Pueden inventar información inexistente con alta confianza.



Sesgos inherentes

Reproducen prejuicios presentes en datos de entrenamiento.



Caja negra

Funcionamiento interno difícil de auditar y explicar.

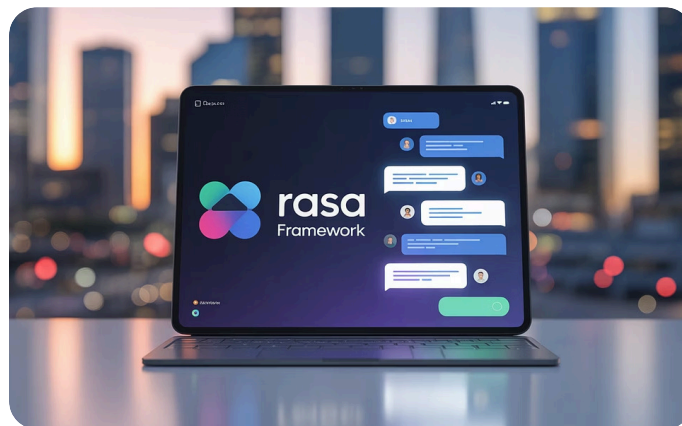
Frameworks y herramientas actuales



LangChain

Framework para desarrollar aplicaciones con LLMs. Facilita la orquestación de componentes.

- Cadenas de procesamiento
- Gestión de memoria
- Integración con herramientas



Rasa con LLMs

Plataforma open-source que ahora integra LLMs como backends conversacionales.

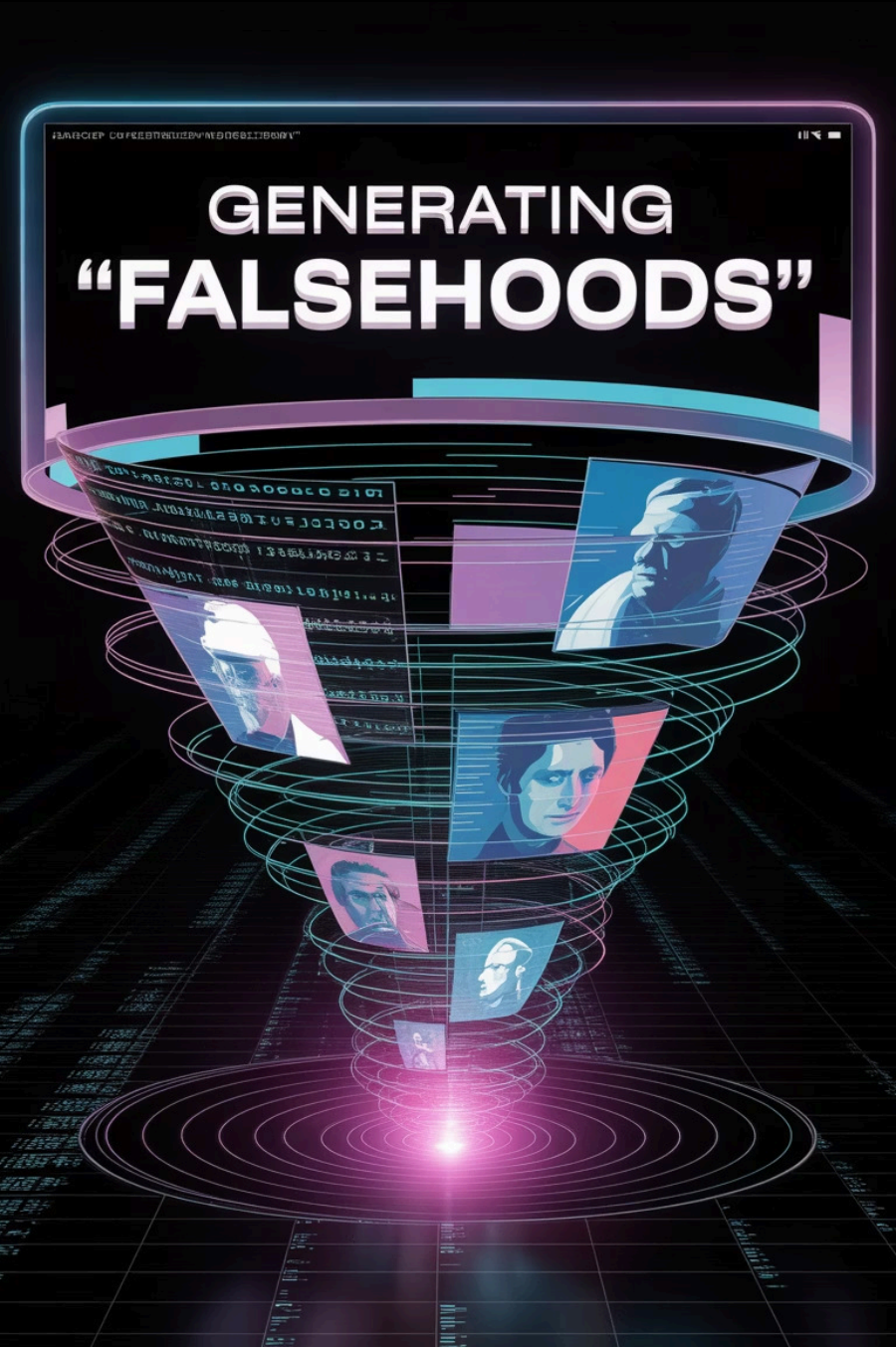
- Control preciso de flujos
- Personalización avanzada



LlamaIndex

Herramienta para indexar y consultar datos externos con LLMs.

- Conexión con fuentes de datos
- Optimizado para RAG



Problemas de los LLMs como chatbots

Alucinación

Generación de información falsa pero aparentemente plausible.

- Citan fuentes inexistentes
- Inventan datos con alta confianza
- Mezclan hechos reales con ficticios

Conocimiento desactualizado

Limitados a información de su entrenamiento.

- Desconocen eventos recientes
- No acceden a datos en tiempo real
- Información obsoleta en dominios cambiantes

Falta de personalización

Dificultad para adaptarse a conocimientos específicos.

- Requieren fine-tuning costoso
- No incorporan documentación privada
- Generalizan demasiado en dominios técnicos

¿Qué es Retrieval-Augmented Generation?

RAG es un **paradigma** que combina la recuperación de información externa con la generación de texto mediante LLMs.

El proceso **enriquece los prompts** con contexto relevante extraído de fuentes confiables.

En lugar de confiar solo en el conocimiento interno del modelo, RAG busca información específica para responder con precisión.

Consulta del usuario

El sistema recibe una pregunta o instrucción.

Recuperación de documentos

Busca información relevante en bases de conocimiento.

Aumento de contexto

Incorpora la información recuperada al prompt.

Generación con LLM

Produce una respuesta basada en el contexto enriquecido.

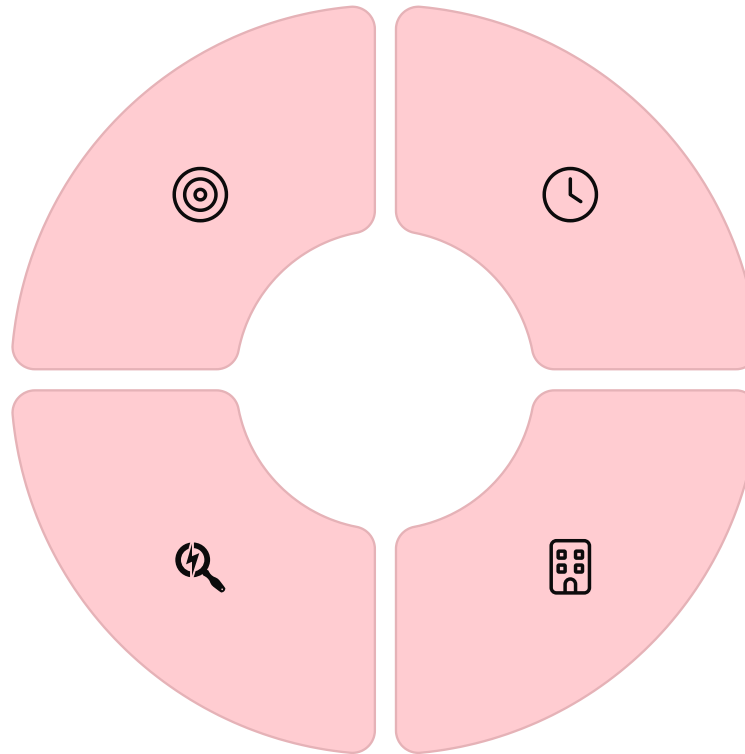
¿Por qué RAG?

Precisión mejorada

Reduce alucinaciones al basar respuestas en fuentes verificables.

Transparencia y citación

Permite rastrear el origen de la información proporcionada.

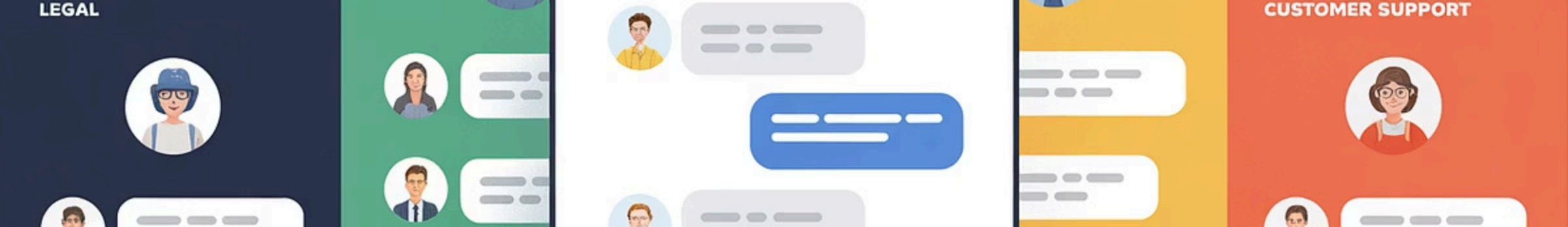


Información actualizada

Accede a documentos recientes que superan la fecha de entrenamiento del LLM.

Conocimiento organizacional

Incorpora documentación interna y conocimiento propietario.



Casos de uso de RAG en chatbots

Legal y compliance

Asistentes que consultan legislación actualizada y jurisprudencia.

- Búsqueda en bases de datos legales
- Acceso a últimas normativas
- Asistencia en interpretación legal

Salud

Chatbots médicos con información clínica actualizada.

- Consulta de protocolos médicos
- Información sobre medicamentos recientes
- Apoyo a diagnósticos basado en literatura

Educación

Tutores virtuales con acceso a materiales específicos.

- Recuperación de textos académicos
- Explicaciones basadas en material de curso
- Generación de ejemplos personalizados

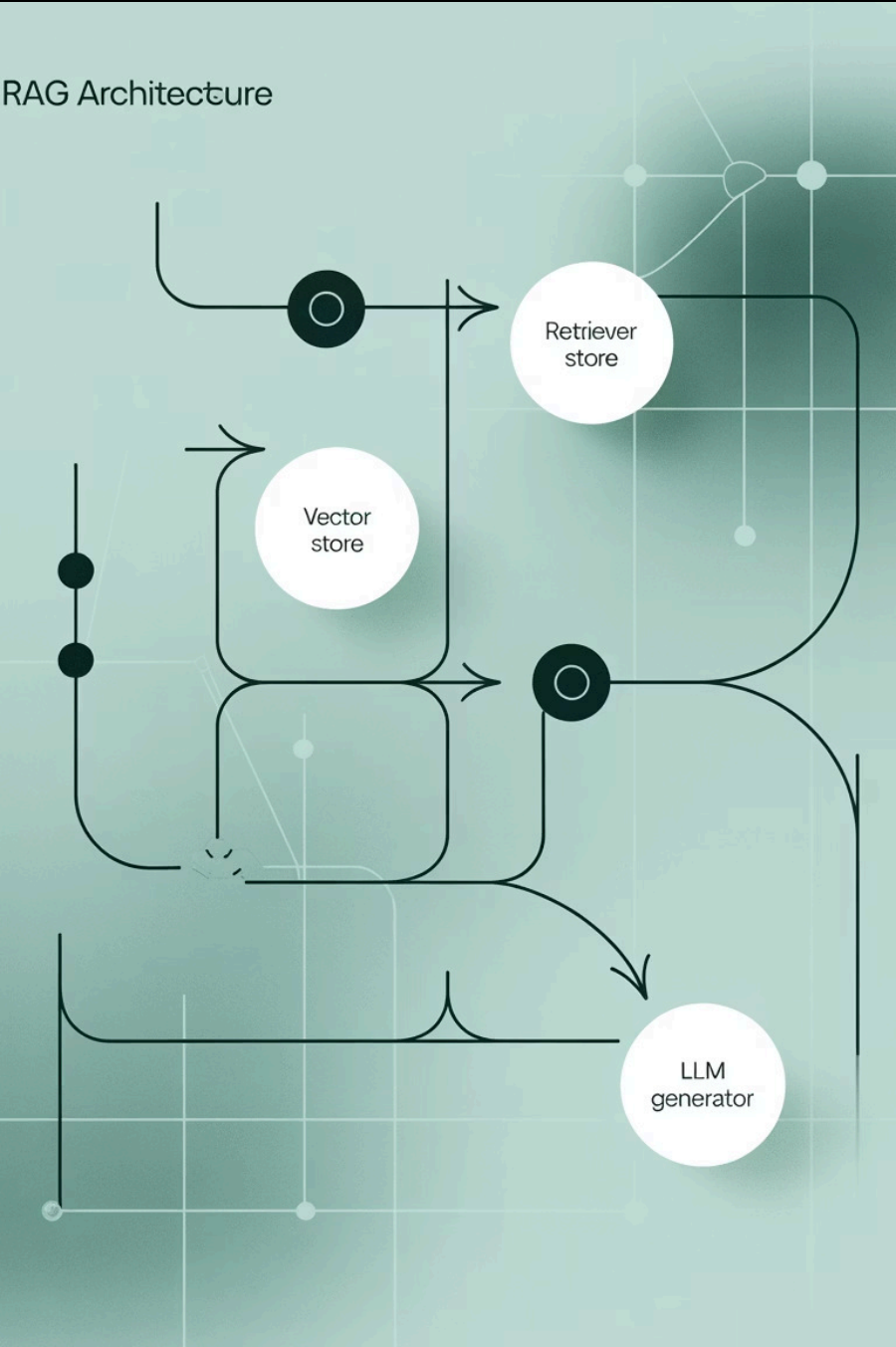
Soporte técnico

Asistentes con acceso a documentación técnica de productos.

- Consulta de manuales técnicos
- Solución de problemas documentados
- Actualización automática con nuevos productos

Comparación: LLM puro vs. RAG

Característica	LLM puro	RAG
Precisión factual	Media (propenso a alucinaciones)	Alta (basada en fuentes)
Información actualizada	Limitada a fecha de entrenamiento	Tan reciente como la base de datos
Personalización	Requiere fine-tuning completo	Adaptable mediante documentos
Latencia	Menor (solo inferencia)	Mayor (búsqueda + inferencia)
Transparencia	Baja (caja negra)	Alta (fuentes verificables)
Costo computacional	Menor	Mayor (infraestructura adicional)



Componentes principales de RAG



Retriever (Sistema de recuperación)

Componente encargado de buscar documentos relevantes basados en la consulta del usuario.

- Transforma consultas en vectores de búsqueda
- Implementa algoritmos de ranking y relevancia
- Filtra y selecciona los mejores candidatos



Vector Store (Base de datos vectorial)

Almacena documentos como representaciones vectoriales para búsqueda semántica eficiente.

- FAISS (Facebook AI Similarity Search)
- Chroma, Pinecone, Weaviate
- Optimizados para búsqueda por similitud



LLM (Generador)

Modelo que genera respuestas utilizando la información recuperada como contexto.

- Fusiona contexto con consulta original
- Sintetiza información de múltiples fuentes
- Mantiene estilo conversacional coherente

Proceso RAG paso a paso



Pregunta del usuario

”¿Cuáles son los efectos secundarios del medicamento X?”



Conversión a embedding

La consulta se transforma en vector mediante modelo de embeddings.



Búsqueda vectorial

Se recuperan documentos similares mediante similitud coseno.



Enriquecimiento de contexto

Se añaden pasajes relevantes al prompt para el LLM.



Generación de respuesta

El LLM sintetiza información y responde en formato conversacional.

Ejemplo gráfico de RAG funcionando

1. Consulta: "¿Qué medidas de seguridad aplica la empresa para datos sensibles?"

El sistema analiza la consulta para identificar intención y entidades clave.

2. Recuperación

- Política de seguridad (similitud: 0.89)
- Manual de protección de datos (similitud: 0.78)
- Protocolos de encriptación (similitud: 0.75)

3. Prompt aumentado

Responde usando la siguiente información:

- Los datos sensibles se cifran con AES-256
- Acceso restringido mediante autenticación 2FA
- Auditorías de seguridad trimestrales

Pregunta: ¿Qué medidas de seguridad aplica...?

4. Respuesta generada

"Nuestra empresa implementa cifrado AES-256 para datos sensibles, requiere autenticación de dos factores para acceso y realiza auditorías de seguridad cada trimestre..."

Implementación con Langchain

Bloques principales

Document Loaders

Cargan documentos desde diversas fuentes (PDF, web, bases de datos).

Text Splitters

Dividen documentos en fragmentos manejables (chunks).

Embeddings

Convierten texto a representaciones vectoriales.

Vector Stores

Almacenan y recuperan vectores eficientemente.

Orquestación

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA
```

```
# Cargar documentos
```

```
loader = PyPDFLoader("documento.pdf")
```

```
documentos = loader.load()
```

```
# Dividir en chunks
```

```
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=1000,
```

```
    chunk_overlap=200
```

```
)
```

```
chunks = splitter.split_documents(documentos)
```

```
# Crear vector store
```

```
embeddings = OpenAIEmbeddings()
```

```
vectorstore = Chroma.from_documents(
```

```
    chunks,
```

```
    embeddings
```

```
)
```

```
# Crear cadena RAG
```

```
qa_chain = RetrievalQA.from_chain_type(
```

```
    llm=OpenAI(),
```

```
    retriever=vectorstore.as_retriever()
```

```
)
```

```
# Consultar
```

```
respuesta = qa_chain.run("¿Pregunta?")
```

Ejemplo mínimo viable en Python

Paso 1: Instalación

```
pip install langchain openai chromadb
```

Paso 2: Configuración

```
import os
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# API key
os.environ["OPENAI_API_KEY"] = "tu-api-key"
```

Paso 3: Implementación RAG

```
# Cargar documento
loader = TextLoader("datos.txt")
documentos = loader.load()

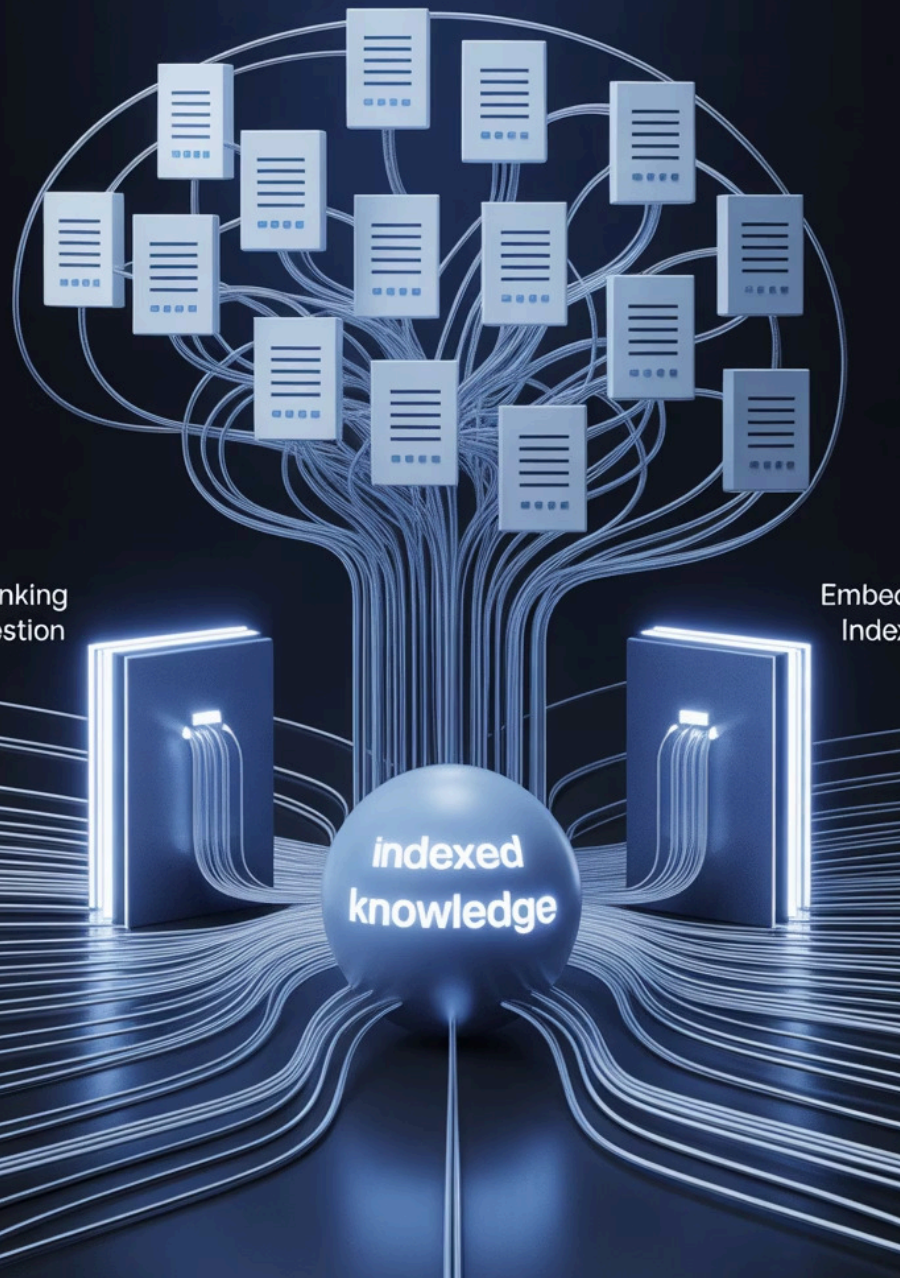
# Dividir texto
text_splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=0
)
textos = text_splitter.split_documents(documentos)

# Crear embeddings y vectorstore
embeddings = OpenAIEmbeddings()
db = Chroma.from_documents(textos, embeddings)

# Crear chatbot RAG
qa = RetrievalQA.from_chain_type(
    llm=OpenAI(temperature=0),
    chain_type="stuff",
    retriever=db.as_retriever()
)

# Consultar
respuesta = qa.run("¿Tu pregunta aquí?")
```

Knowledge Base Construction



Construcción de base de conocimiento

Recopilación de documentos

Identificar y reunir fuentes relevantes para el dominio del chatbot.

- Manuales técnicos y documentación
- Bases de conocimiento existentes
- FAQs y recursos de soporte

Preprocesamiento

Limpiar y normalizar el texto para optimizar la indexación.

- Eliminar formatos y metadatos innecesarios
- Corregir errores tipográficos y estructurales
- Normalizar términos y abreviaturas

Chunking (fragmentación)

Dividir documentos en unidades manejables para búsqueda eficiente.

- Determinar tamaño óptimo de fragmentos (500-1000 tokens)
- Mantener coherencia semántica en fragmentos
- Añadir solapamiento para evitar pérdida de contexto

Embedding e indexación

Generar vectores y almacenarlos en base de datos optimizada.

- Seleccionar modelo de embedding adecuado
- Configurar vector store con parámetros óptimos
- Implementar estrategias de actualización

Desafíos comunes en RAG

Latencia

El proceso de recuperación aumenta tiempo de respuesta.

- Indexación optimizada
- Caché de resultados frecuentes
- Búsqueda asíncrona

Grounding (anclaje)

Asegurar que respuestas se basen en documentos recuperados.

- Prompts específicos de seguimiento
- Citación de fuentes
- Penalización de contenido no fundamentado

Relevancia

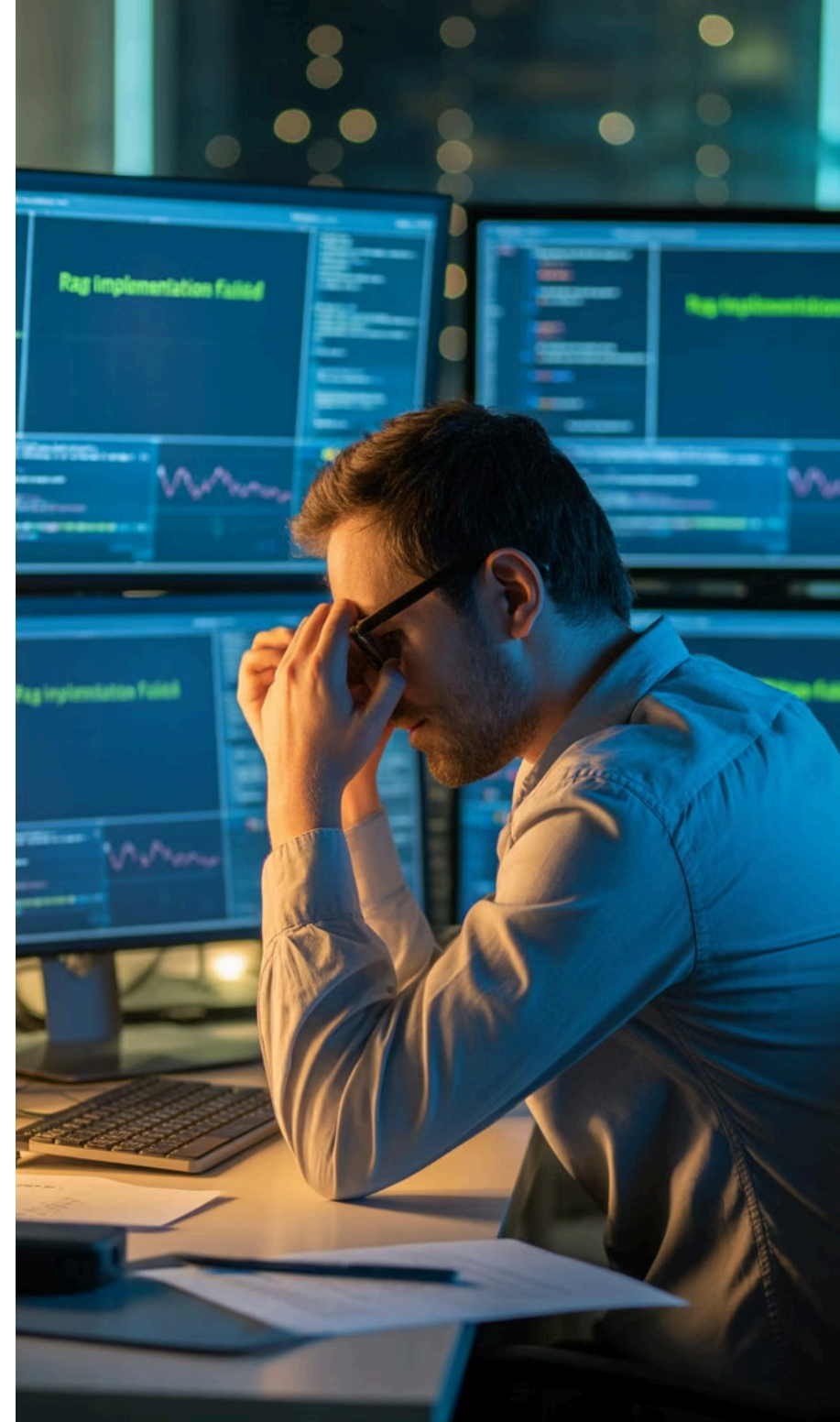
Recuperar documentos verdaderamente útiles para la consulta.

- Reranking semántico
- Expansión de consultas
- Feedback de relevancia

Seguridad

Protección de datos sensibles y prevención de filtraciones.

- Control de acceso granular
- Filtrado de contenido sensible
- Auditoría de consultas y respuestas



Futuro de los chatbots con RAG

Chatbots multiagente

Sistemas con múltiples agentes especializados que colaboran para resolver problemas complejos.

- Agentes con roles específicos
- Coordinación mediante meta-agentes
- Especialización por dominios de conocimiento

Conexión con herramientas

Chatbots que interactúan con sistemas externos para ampliar capacidades.

- Navegación web en tiempo real
- Consultas a bases de datos
- Integración con CRMs y ERPs
- Ejecución de código y análisis



Personalización avanzada

Chatbots que aprenden preferencias específicas de cada usuario.



Ética y transparencia

Mecanismos de explicabilidad y controles de sesgo integrados.



Especialización por industria

Asistentes optimizados para dominios específicos como legal, salud o ingeniería.